

Topic 1 — Data Ingestion, Real-Time Stream Processing & Enterprise Monitoring Platform

Detailed Architecture, Design, Operations and Interview/Presentation Guide

Prepared: September 2026

How to use this guide

This document expands each presentation slide into a speaker-ready explanation. It is designed for architecture discussions, technical interviews, customer presentations and production design reviews.

1. Executive Overview

Design an enterprise platform that continuously ingests telemetry and business events, processes streams in near real time, correlates signals, and presents actionable operational intelligence.

Core flow: Sources → Ingestion → Buffer/Stream → Processing → Correlation/Analytics → Monitoring/ITSM/ChatOps/Automation.

2. Business Problem

Enterprises generate logs, metrics, traces, application events, network events, cloud telemetry and business events at high volume. Point-to-point integrations create brittle dependencies, duplicated processing, alert storms and slow incident response.

The target state is an event-driven platform with standardized schemas, scalable ingestion, durable buffering, centralized observability and automated response.

3. Reference Architecture

Sources: applications, servers, Kubernetes, databases, network devices, AWS/Azure/GCP services, SaaS platforms.

Ingestion: agents/collectors, APIs, webhooks, syslog, OpenTelemetry, Kafka-compatible producers.

Platform: Kafka/Pub/Sub/Event Hubs/Kinesis, stream processors, schema registry, data lake/search store, metrics backend, observability UI, ITSM and automation.

4. Data Ingestion Patterns

Pull: scheduled APIs and exporters. Push: webhooks, agents and application events. Batch: files/object storage. Streaming: continuous event publication.

Use a common envelope containing event_id, timestamp, source, environment, tenant, entity/CI, event_type, severity and correlation identifiers.

5. Ingestion Reliability

Use buffering, acknowledgements, retries with exponential backoff, dead-letter queues, replay capability and idempotent consumers.

Define delivery semantics explicitly: at-most-once, at-least-once or effectively-once. Monitor lag, throughput, error rate and rejected messages.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

6. Real-Time Stream Processing

Typical stages: parse → validate → enrich → filter → aggregate → correlate → route.

Windowing supports tumbling, sliding and session windows. Stateful processing can maintain counters, baselines, deduplication keys and correlation state.

7. Event Correlation

Correlation combines multiple low-level events into one operational story. Example: CPU saturation + pod restarts + latency spike + database connection errors may represent one service degradation.

Correlation keys can include service, host, pod, namespace, trace ID, deployment ID, change ID and time proximity.

8. Enterprise Monitoring Platform

A mature platform unifies metrics, logs, traces, events, topology and service ownership. It should provide dashboards, alert rules, SLOs, dependency views, incident context and auditability.

Separate signal collection from decisioning so ingestion remains stable even when downstream consumers change.

9. Observability Data Model

Metrics answer how much/how fast; logs explain what happened; traces show request paths; events describe state changes. A useful platform normalizes all signals around entities such as service, host, cluster, namespace and business application.

10. Alert Management & Noise Reduction

Apply deduplication, suppression, maintenance windows, threshold hysteresis, dependency-aware suppression and event aggregation.

Distinguish duplicate alerts from false alerts: duplicates are repeated representations of the same signal; false alerts are signals that do not represent a meaningful condition.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

11. Monitoring Kubernetes

Collect node, pod, container, workload and application telemetry. Key signals include CPU/memory saturation, restarts, pending pods, scheduling failures, API-server health, control-plane latency, network errors and application SLOs.

Use labels carefully; uncontrolled cardinality can make observability systems expensive and slow.

12. Scalability & HA

Scale collectors horizontally, partition streams by stable keys, use replicated brokers and multi-zone deployments. Keep hot operational data in fast stores and move historical data to lower-cost storage.

Plan capacity using events/sec, bytes/sec, peak multiplier, retention, consumer count and recovery time.

13. Security & Governance

Use TLS in transit, encryption at rest, workload identity/RBAC, secret management, tenant isolation and least privilege. Classify sensitive telemetry and redact secrets/PII before broad distribution.

Maintain audit logs for configuration changes, access and automated actions.

14. Operational KPIs

Measure ingestion throughput, processing latency, consumer lag, dropped events, alert precision, alert volume, MTDD, MTTA, MTTR, automation success rate, platform availability and cost per million events.

Use SLOs for the monitoring platform itself.

15. Production Best Practices & Example

Example: A payment platform publishes application events and telemetry into a durable stream.

Processing enriches events with CMDB ownership and topology, correlates related signals, creates one actionable incident, posts context to ChatOps and triggers safe remediation.

Best practices: schema governance, replayability, idempotency, backpressure, observability of the observability platform, controlled automation and continuous tuning.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

Reference Areas

- [OpenTelemetry documentation](#)
- [Apache Kafka documentation](#)
- [Prometheus documentation](#)
- [Kubernetes documentation](#)
- [AWS Well-Architected Framework](#)

Note: Product capabilities and configuration details vary by cloud/provider version. Validate implementation details against the current official product documentation before production rollout.